

Schema and Data Integration

Knowledge Objectives

1. Identify the problem of information integration
2. Enumerate the three solutions to information integration
3. Explain the three characteristics of a distributed database
4. Name five kinds of system heterogeneities
5. Name ten kinds of semantic heterogeneities

Understanding Objectives

1. Translate a global query into the corresponding local ones using GAV mappings
2. Given a similarity and merge functions, eliminate the duplicates in a table using the R-Swoosh algorithm

Application Objectives

1. Given two schemas of the same domain with structural discrepancies, decide whether they represent the same reality or not
2. Given a schema, propose an equivalent alternative showing any kind of semantic heterogeneity

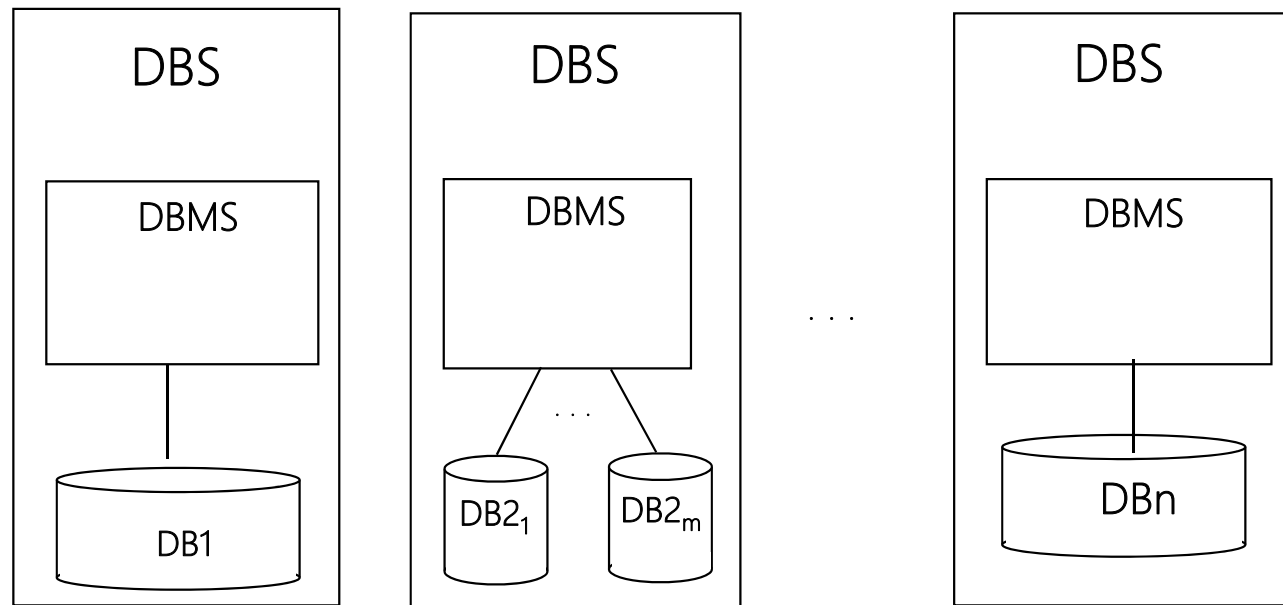
Problem Definition

The problem (I)



?

Answer a query that requires accessing several databases



The problem (II)

It is not:

- BD connectivity (assumed)
- Electronic Data Interchange (EDI), Business to Business (B2B), eB-XML (e.g., SOAP)
- Remote database access
- Multiclient/Multiserver architecture
- Distributed DBMS

It is about being able to pose **one query**, and get **one answer**, so that in the preparation of the answer data coming from **several DBs** is processed.

Football players example

- FIFA
 - 19k+ players, 100+ attributes extracted from the latest edition of FIFA
 - Player name example: Lionel Andrés Messi Cuccittini
- FBref
 - 2,699 players, 230 attributes extracted from FBref sports statistics website
 - Player name example: L. Messi
- TransferMarkt
 - ~28,000 players, 23 attributes scraped from Transfermarkt website
 - Player name example: Lionel Messi

<https://www.kaggle.com/datasets/stefanoleone992/fifa-22-complete-player-dataset>

https://www.kaggle.com/datasets/goaltergeist/tidy-fbref-2021-player-data-credit-ronanmann?select=fb-data_2020-21_players_share.csv

<https://www.kaggle.com/datasets/davidcariboo/player-scores>

Flights aggregator example

TABLE 1.1: Sample data for Airline1.Schedule

	FI	FN	SD	ED	DT	DA	AT	AA
r ₁₁	123	49	2013-10-01	2014-03-31	18:05	EWK	21:10	SFO
r ₁₂	234	49	2014-04-01	2014-09-30	18:20	EWK	21:25	SFO
r ₁₃	345	55	2013-10-01	2013-10-01				
r ₁₄	346	55	2013-10-01	2013-10-01				

TABLE 1.2: Sample data for Airline2.Schedule

	FI	DD	DT
r ₂₁	123	2013-12-21	18:4
r ₂₂	123	2013-12-28	21:3
r ₂₃	345	2013-12-2	1
r ₂₄	346	2013-12-2	2:35

TABLE 1.3: Sample data for Airfare4.Flight

	FI	FN	DA	DT	AA	AT
r ₆₃	458	A1-49	Newark Liberty	2014-04-12	18:05	San Francisco
r ₆₄	4		Newark Liberty		23:35	
r ₆₅	4		Newark Liberty		23:35	
r ₆₆	4		Newark Liberty		00:00 (+1d)	
r ₇₁						
r ₇₂						
r ₇₃						

TABLE 1.4: Sample data for Airport3.Departures

	AL	FN	S	A	GT
r ₄₁	A1	49	2013-12-21	2013-12-21	18:45 18:53 C 98 2
r ₄₂	A1	49	2013-12-28	2013-12-28	21:29 21:38 C 101 2

TABLE 1.5: Sample data for Airinfo5.AirportCodes, Airinfo5.AirlineCodes

Airinfo5.AirportCodes		Airinfo5.AirlineCodes	
AC	AN	ALC	ALN
r ₈₁	EWK Newark Liberty, NJ, US	r ₉₁	A1 Airline1
r ₈₂	SFO San Francisco, CA, US	r ₉₂	A2 Airline2

TABLE 1.6: Sample data for Airfare4.Fares

	ADT	AA	SAD	SAT	AAT
r ₅₁	16:00	EWK	2013-12-21	23:35	00:15 (+1d)
r ₅₂	16:15	EWK	2013-12-22	23:35	00:30
r ₅₃	16:05	EWK	2013-12-29	00:05	23:57 (-1d)
r ₅₄	16:05	EWK	2013-12-21	20:05	20:45
r ₅₅	16:05	EWK	2013-12-22	11:00	10:59

TABLE 1.7: Sample data for Airfare4.Fares

	GT	LT	T	G	R
r ₅₁	A2	53	2013-12-21	2013-12-22	00:21 00:15 B 53 2
r ₅₂	A2	53	2013-12-22	2013-12-23	00:40 00:30 B 53 2
r ₅₃	A1	55	2013-12-29	2013-12-29	23:35 23:31 C 101 1
r ₅₄	A2	49	2013-12-21	2013-12-21	20:50 20:45 B 55 2

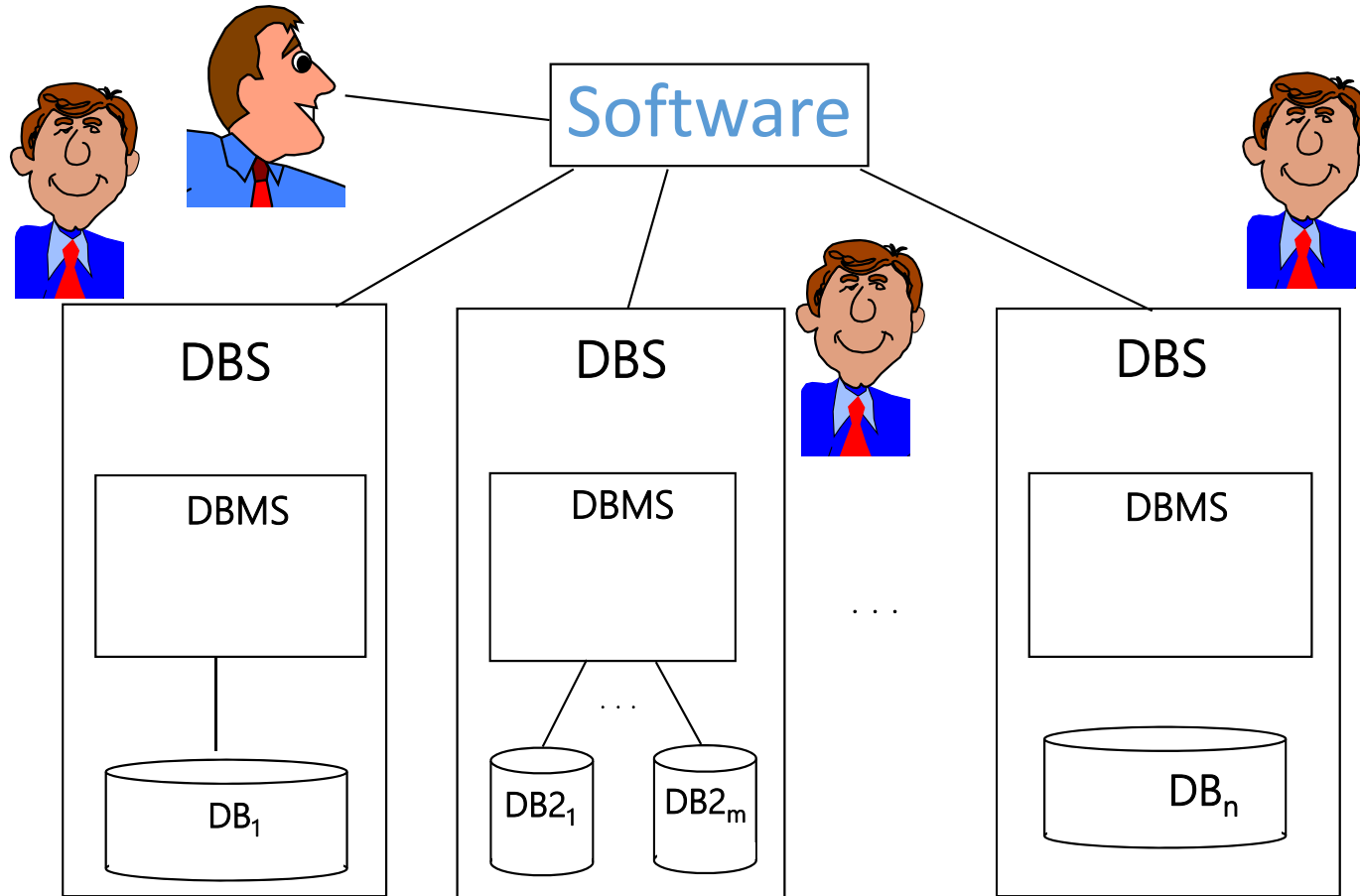
Hundreds of airlines
Thousands of airports

X. L. Dong and D. Srivastava

Solutions

- a) Manually query the different databases separately
 - Know the available databases
 - Data
 - Data model
 - Query language
 - Decompose the query
 - Integrate the results
- b) Create a new database containing all necessary data
 - Design it
 - Move data
 - Modify the applications to use the new repository
 - Test everything
- c) Build a software layer on top of the databases that automatically splits the queries and integrates the answers
 - Automate the processing of the queries
 - Add a new software layer that defines two access levels

Users in the integrated system



Distributed Database

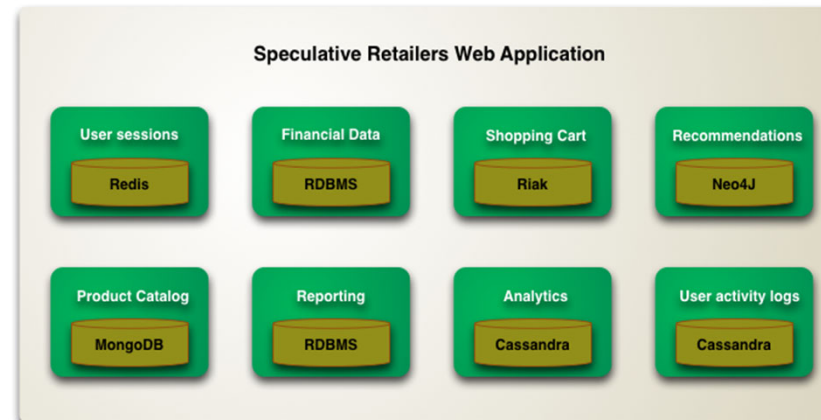
"A distributed database (DDB) is a collection of **multiple, logically interrelated databases** (known as nodes or sites) **distributed over a computer network**. A distributed database management system (DDBMS) is thus, the software system that permits the management of the distributed database and makes the distribution **transparent to the users**."

Tamer Özsu & P. Valduriez

Principles of DDB Systems, Springer, 2011

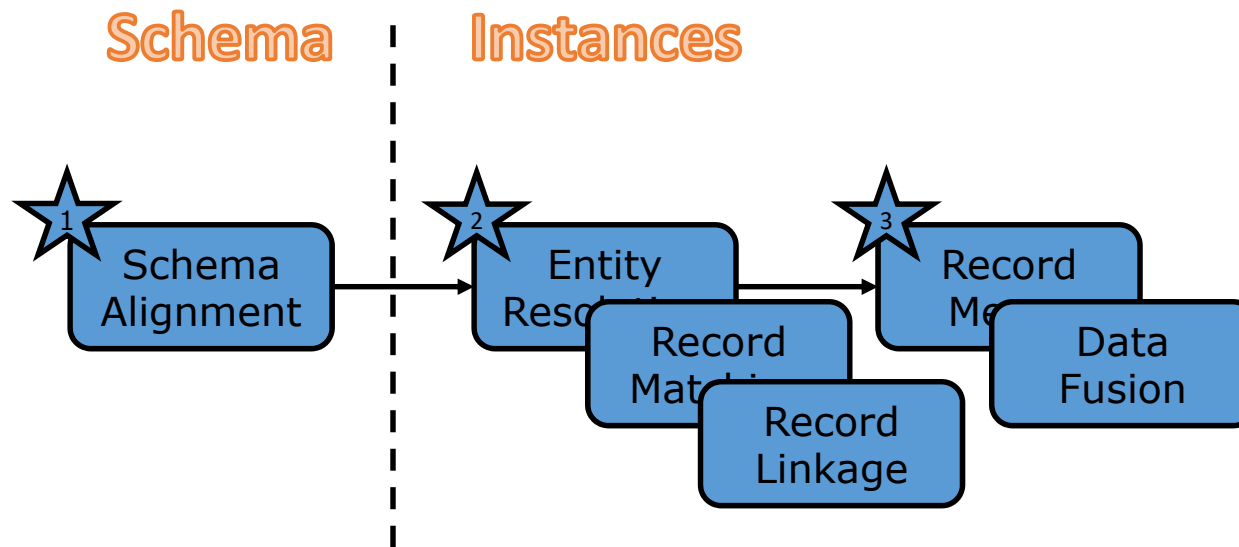
Classification of Distributed Databases

- Autonomy
 - a) Design
 - b) Execution
 - c) Association
- Heterogeneity
 - a) System
 - b) Semantic



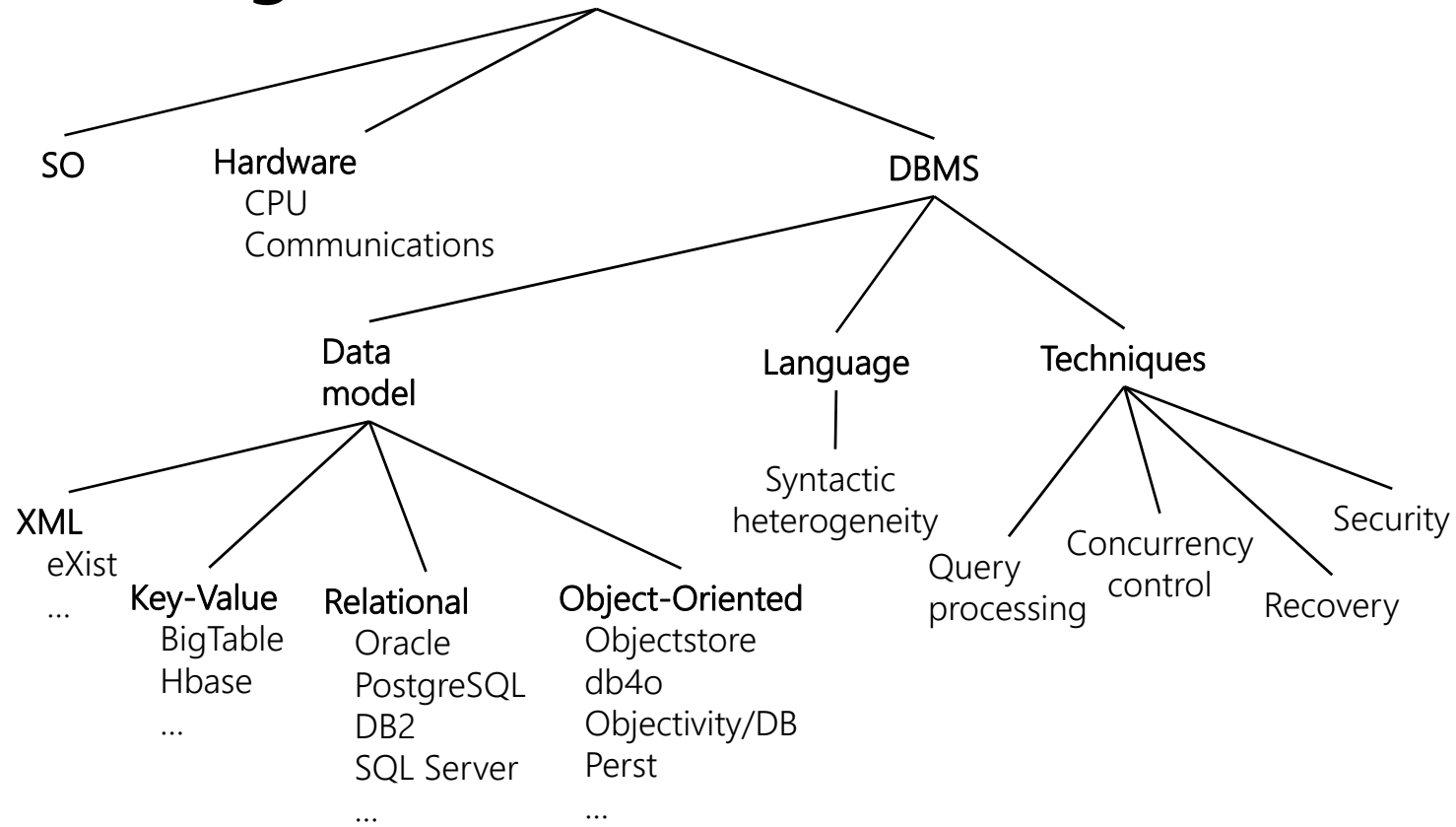
Polyglot persistence, Martin Fowler

Major steps to overcome semantic heterogeneities



Kinds of Heterogeneity

System heterogeneities

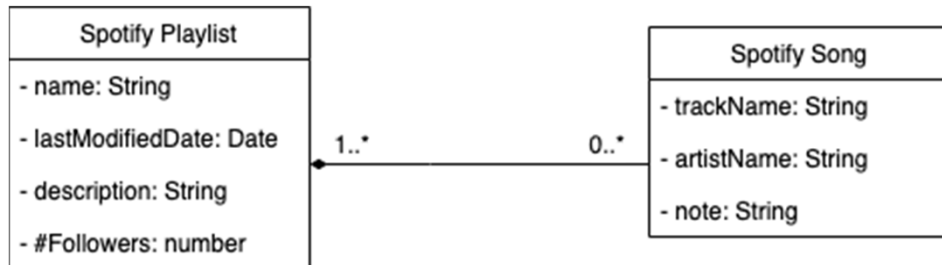


Mars Climate Orbiter

- What: Space probe
- Who: NASA
- Where: Mars
- When: 1999
- Objective: Landing
- Result: Crashing
- Why:
 - Impulse software produced results in imperial units
 - I.e., pound-force seconds
 - Trajectory software calculating the position expected values in the metric system
 - I.e., newton-seconds
- Cost: 327.6 million US\$

Schema and instances

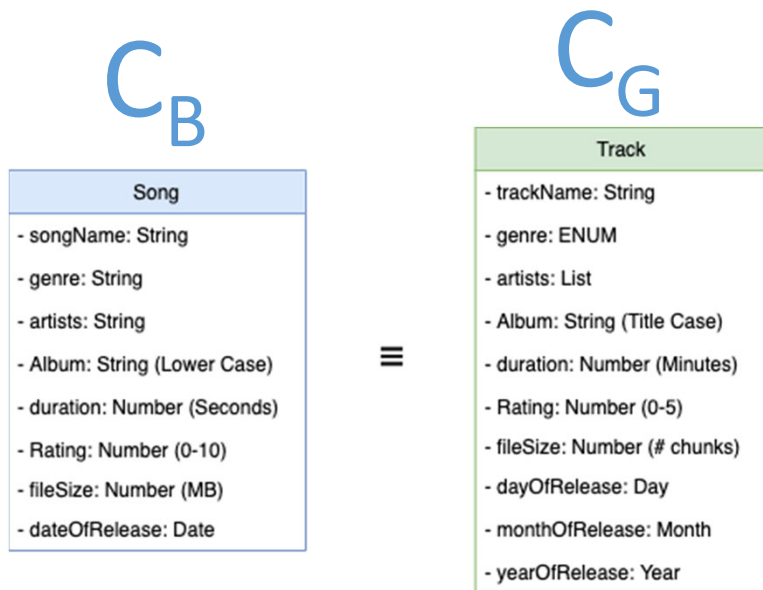
Schema



Instance

```
{
  "playlists": [
    {
      "name": "Bjork Pitchfork",
      "lastModifiedDate": "2018-10-16",
      "tracks": [
        {
          "trackName": "Cosmogony",
          "artistName": "Bjork",
          "note": "Usually the first song of the concert"
        },
        {
          "trackName": "Hunter",
          "artistName": "Bjork",
          "note": null
        },
        ...
      ],
      "description": null,
      "numberOfFollowers": 0
    },
    ...
  ]
}
```

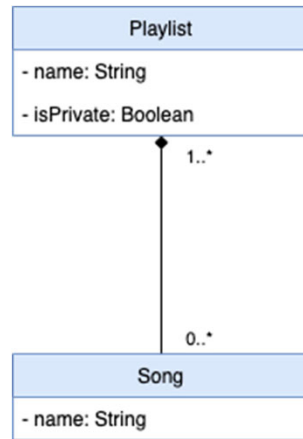
Intra-class heterogeneities



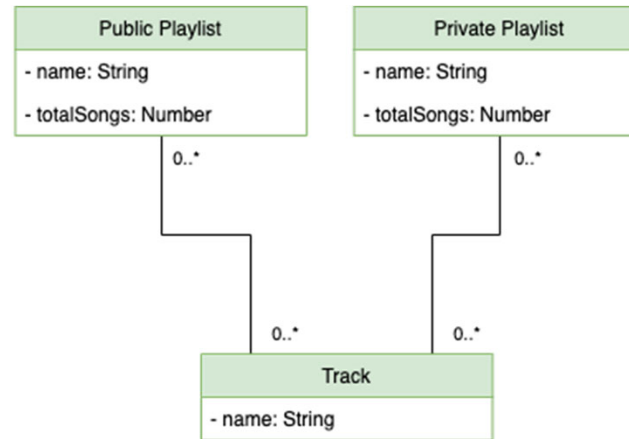
Class	Naming		When the classes have different names (e.g., Song in C_B and Track in C_G).
Attribute	Naming		When the attributes have different names (e.g., songName and trackName).
	Type		When two equivalent attributes have different type representation between domains (e.g., genre in C_B and in C_G).
	Single/Multi-valuation		When the attribute is single-valued in one domain but multi-valued in another (e.g., artists in C_B and C_G).
	Representation	Format	When the attributes have the same type, but are represented in different formats (e.g., Album in C_B and C_G).
		Measure	When two attributes are numerical, but the unit to represent them differs (e.g., duration in C_B and C_G).
		Scale	When two attributes are numerical, but the scale to represent them differs (e.g., rating in C_B and C_G).
		Dimension	When two attributes are numerical, but the dimension to represent them differs (e.g., fileSize in C_B and C_G).
Composition		When one attribute is the result of the composition of many attributes (e.g., dateOfRelease in C_B and dayOfRelease, monthOfRelease and yearOfRelease in C_G).	

Inter-class heterogeneities

C_B



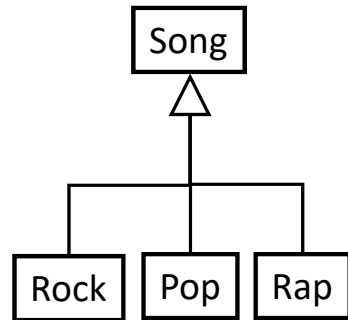
C_G



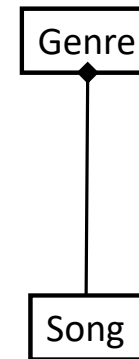
Super/Subclass of		When a Class in a schema is a generalization or specification of a class in another (e.g., Playlist in C_B and Public/Private Playlist in C_G).
Aggregation	Composition	When a class is composed by an aggregation of another class in one domain, but is an independent class in another (e.g., Song is aggregated inside Playlist in C_B while Track has simple associations with Public/Private Playlist in C_G).
	Derivation	When the characterization of a class is derived from the aggregation of another (e.g., Public/Private Playlist in C_G have totalSongs, an attribute defined by the aggregation of Track in C_G . While in C_B such dependency is absent).

Data-Metadata heterogeneity

```
{  
  "rock": [ ... ],  
  "pop": [ ... ],  
  "rap": [ ... ]  
}
```



Vs

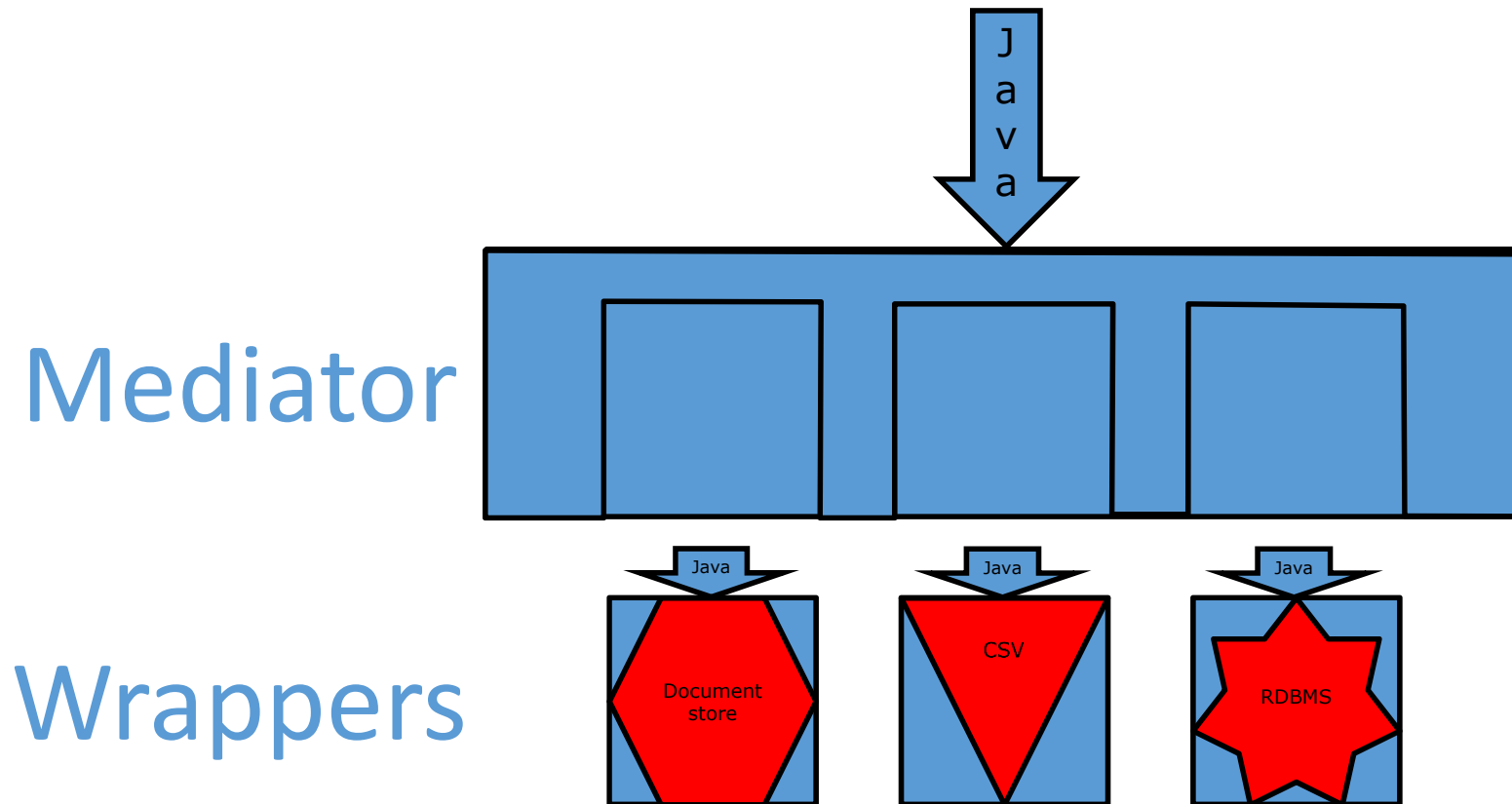


```
{  
  "genre": [  
    { "name": "Rock", ... },  
    { "name": "Pop", ... },  
    { "name": "Rap", ... },  
  ]  
}
```

These two alternatives allow to represent basically the same reality, but what is considered metadata (i.e., subclasses of "Song") at LHS is considered data (i.e., instances of "Genre") at RHS

Overcoming System Heterogeneity

Wrappers-Mediator Software Architecture

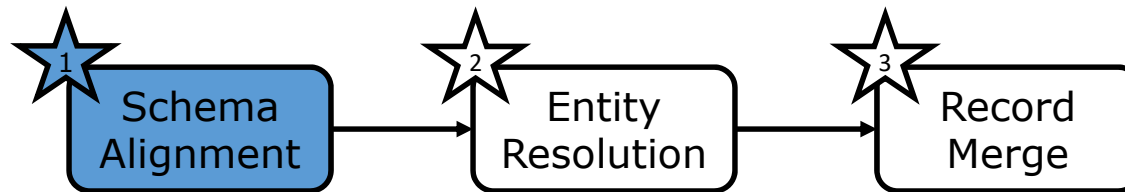


Wrapper example (CSV→Java)

```
public ArrayList<Player> queryPlayers() {
    BufferedReader br = null;
    try {
        br = new BufferedReader(new FileReader("Players.csv"));
        objects List<Player> empList = new ArrayList<Player>();
        String line = "";
        br.readLine();
        while ((line = br.readLine()) != null) {
            String[] PlayerDetails = line.split(",");
            if(PlayerDetails.length > 0 ) {
                Player pl = new Player(
                    Integer.parseInt(PlayerDetails[0]),
                    PlayerDetails[1],
                    PlayerDetails[2],
                    Integer.parseInt(PlayerDetails[3]));
                empList.add(pl);
            }
        }
        return empList;
    }
    catch(Exception ee) {
        throw new MyException(e);
    }
}
```

Wrapper example (SQL→Java)

```
public ResultSet RetrieveTable () throws MyException {
    try {
        Statement stmt = getConnection().createStatement();
        return stmt.executeQuery("
            SELECT
                X.A AS player_name,
                1960+Y.B AS yearOfBirth,
                CASE WHEN Z.C = "Fotbal Club Barselona"
                THEN "FCB"
                ELSE Z.C END AS club
            FROM X, Y, Z
            WHERE X.FK=Y.ID AND Y.FK=Z.ID;
        ");
    } catch (Exception e) {
        throw new MyException(e);
    }
}
```



Schema integration

Schema alignment and mapping

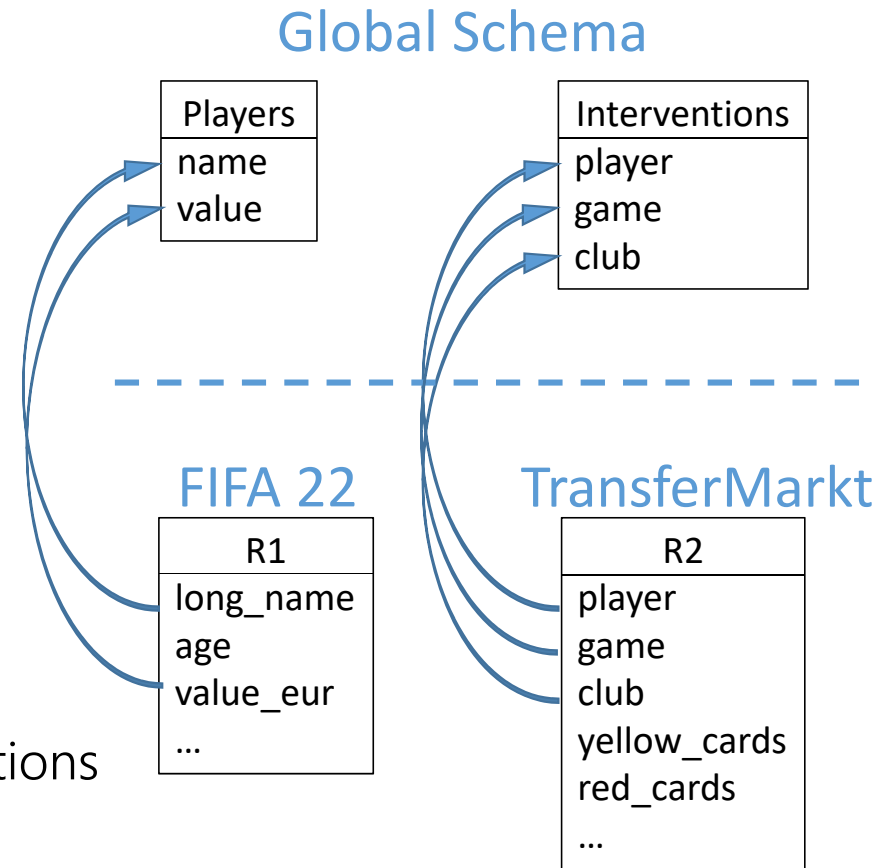
Alignment Process

Input

- Schemas of the different data sources

Output

- Global/Integrated/Mediated schema
- Attribute matching
 - Between sources and global
- Schema mapping
 - Specify semantic relationships & transformations



Mappings

- Kinds

- *Sound*

- $q_{\text{Obtained}} \subseteq q_{\text{Desired}}$ (e.g., $R1 \subseteq \text{Players}$)

- *Complete*

- $q_{\text{Obtained}} \supseteq q_{\text{Desired}}$ (e.g., $R2 \supseteq \text{Interventions}$)

- *Exact*

- $q_{\text{Obtained}} = q_{\text{Desired}}$

- Techniques (to be implemented in the mediator)

- Global As View (GAV)
 - Local As View (LAV)
 - Global/Local As View (GLAV)
 - Peer To Peer (P2PDBMS)

Example of GAV (I)

- Global schema
 - Players (name, value)
 - Interventions (player, game, club)
- Local schemas
 - FIFA 22
 - R1 (long_name, age, value_eur, ...)
 - TransferMarkt
 - R2 (player, game, club, yellow_cards, red_cards, ...)
- Mappings
 - Players (n, v):= R1[long_name, value_eur]
 - Interventions (p, g, c):= R2[player, game, club]

Example of GAV (II)

$q(t, r) := \text{Players } [n, v] * \text{Interventions } [n, c='FCB']$



$\text{Players } (n, v) := R1[\text{long_name}, \text{value_eur}]$

$q(t, r) := R1 [n, v] * \text{Interventions } [n, c='FCB']$



$\text{Interventions } (p, g, c) := R2[\text{player}, \text{game}, \text{club}]$

$q(t, r) := R1 [n, v] * R2 [n, c='FCB']$

Query translation

Query execution

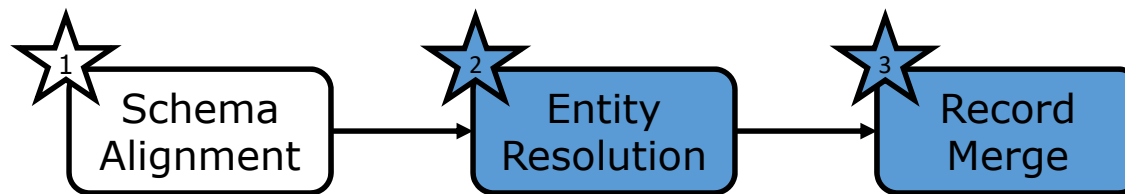
2
FIFA 22



1
TransferMarkt

Characteristics of GAV

- The global schema is defined as a view over the data sources (bottom-up design)
- A mapping must be defined for each element in the global schema
- Query processing is absolutely equivalent to view processing in a centralized DBMS
- It can result too rigid
 - Any change in the data sources affects the global schema
 - Mappings (a.k.a., queries) need to be redefined



Data integration

Record matching and merging

Integration

1. Entity Resolution (Record matching)
 - Find an Object Identification Function

2. Record merging
 - Schemas
 - Codification
 - Granularity
 - Units and scales
 - Data and metadata

Entity resolution (Object identification)

“Decide whether two tuples correspond to the same object in the real world.”

- Problems to face:
 - Misspellings
 - Variant names
 - Misunderstanding of names
 - Evolution of values
 - Abbreviations
- Simplifications
 - Normalize the strings
 - Compare field by field

Comparison/Similarity/Identification function ($\approx$)

- Distance-based
 - Hamming distance
 - Levenshtein distance
 - Edit distance
 - Jaccard similarity
 - $\frac{|A \cap B|}{|A \cup B|}$
- Rule-based
- Probabilistic
- Classification-based

Merge function ($\wedge$)

- Solutions in case of different values
 - Generate null
 - Generate a multi-valued
 - Choose one based on source trustworthiness
 - Use crowdsourcing

R-Swoosh algorithm

```
⇒ O := ∅;  
  while (I <> ∅) do {  
    ⇒ pick up r ∈ I;  
      if (∃ s ∈ O so that s ≈ r) {  
        ⇒ I := I - r; O := O - s; I := I ∪ {r ∧ s};  
      } else {  
        ⇒ I := I - r; O := O ∪ {r};  
      }  
  }
```

$s \approx r \Leftrightarrow \text{Jaccard}(s, r) = \frac{|r \cap s|}{|r \cup s|} \geq 0.3$
 $r \wedge s = r \cup s$
 $I = \{ \text{abc} \text{ ac} \text{ bc} \text{ cd} \}$
 $O = \{ \quad \quad \}$

Closing

Summary

- Distributed databases classification
- Heterogeneities
 - Semantic
- Quality measures:
 - Accuracy
 - Completeness
 - Consistency
 - Timeliness
- Quality rules
- Entity resolution (Object identification)
 - R-swoosh algorithm
- Record merging

References

- O. A. Bukhres and A. K. Elmagarmid (Eds.). *Object-Oriented Multidatabase Systems*. Prentice-Hall, 1996
- A. Maydanchik. *Data Quality Assessment*. Technics Publications, 2007
- J. Bleiholder and F. Naumann. Data Fusion. *ACM Computing Surveys*, 41(1), 2008
- C. Batini, et al. *Methodologies for data quality assessment and improvement*. *ACM Computing Surveys*, 41(3), 2009
- H. Garcia-Molina et al. *Database Systems*. Prentice Hall, 2009
- T. Özsu and P. Valduriez. *Principles of Distributed Database Systems*. Springer, 2011
- X. L. Dong and D. Srivastava. *Big Data Integration*. Morgan&Claypool, 2015
- C. Batini and M. Scannapieco. *Data and Information Quality*. Springer, 2016